

STRINGS IN C++

A beginner-friendly walk-through of how characters become strings, how strings live in memory, and the everyday tools you'll use to bend them to your will.

devweekends.com

resources.devweekends.com

TODAY'S PLAN

01 Characters & memory basics

The `char` type, byte sizes, and the four ASCII numbers worth memorising.

02 What a string really is

Definition, the null terminator `'\0'`, and how a string sits in memory.

03 Working with strings

Two ways to initialize, modifying characters, and the size-vs-length gotcha.

04 Reading & printing in C++

Using `cin`, `cin.getline`, and `cout` — pure C++ idioms only.

05 Practice + Q&A

Two hand-picked beginner problems to lock the concepts in.

A 5-SECOND RECAP

ARRAYS + THE CHAR TYPE

An **array** is a box that holds many values of the **same type**, lined up in memory and accessed by an index starting at **0**. Use the `char` type for letters — written inside **single quotes**.

! **char = 1 byte (8 bits) → 256 values**

Plenty of room for letters, digits, and symbols.

! **int = 4 bytes (32 bits)**

A character is **4× smaller** than an int. Strings are cheap.

```
// Arrays of different types
int    nums[5];      // box of 5 ints
float  price[5];     // box of 5 floats
char   letters[5];   // box of 5 chars

// One character – single quotes
char grade = 'A';

sizeof(grade); // → 1 byte
sizeof(int);   // → 4 bytes
```

EVERY CHAR IS A NUMBER

A

65

Z → 90 (A + 25)

a

97

Z → 122 (A + 25)

'0'

48

'9' → 57

\0

0

NULL TERMINATOR

Difference between any uppercase letter and its lowercase form is exactly **32**. So

`'a' - 'A' = 32` — a tiny fact that solves a surprising number of string problems.

ASCII · THE BIG FOUR

IN ONE LINE

A STRING IS AN ARRAY OF CHARS

A C++ C-style string is a **char array** that ends with the special **null character** `'\0'`.

Think of `'\0'` as a "stop sign" — it tells the compiler where the word ends, no matter how big the array is.



7 visible letters + 1 invisible terminator =
total size 8 in memory.

LONG WAY VS SHORT WAY

```
// Long way - list each character
char str[8] = {
    'c', 'o', 'l', 'l',
    'e', 'g', 'e', '\0'
};
// You must add '\0' yourself
```

```
// Short way - use double quotes
char str[] = "college";

// Compiler auto-adds '\0' at the end
// Auto-calculates size = 8
// 99% of real code uses this style
```

Both produce the exact same bytes in memory. The short form is friendlier and harder to mess up — that's what we'll use from here on. Each character is **editable by index**: `str[0] = 'M';`

TINY BUT POWERFUL

THE STOP SIGN

`'\0'` is a single character with ASCII value **0**. It's invisible when printed, but every string function watches for it.

If it's missing or misplaced, your string keeps printing memory garbage until it randomly hits a zero byte.

01

Loop until you hit it

`while (str[i] != '\0')` — the universal "iterate a string" pattern.

02

`cout << str` uses it

Prints character by character, stops when it sees the terminator.

03

Always reserve one slot

A word of **N** letters needs an array of size **N + 1**.

SIZE, LENGTH & WRONG SIZES

01

Size = total slots

`sizeof(str)` — includes the `'\0'`. For `"college"` → 8.

FROM THE ARRAY

02

Length = visible chars

`strlen(str)` — excludes the `'\0'`. For `"college"` → 7.

FROM THE CONTENT

03

Always reserve N + 1

Plan for the terminator. `char s[7] = "college";` is too small · `char s[20] = "college";` works fine (extra slots unused).

RULE OF THUMB

Too small → compile error or garbage on print. Too big → harmless, just wastes a few bytes. When in doubt, declare bigger.

PRINTING IN C++

01 Manual loop · one character at a time

`for (int i = 0; str[i] != '\0'; i++) cout << str[i];` · longest, but shows what's happening underneath.

02 `cout << str` · the one-liner

Pass the array name. `cout` follows the terminator and stops on its own — same idea as method 01, just written for you.

03 Add a newline

`cout << str << endl;` or `cout << str << "\n";` — useful when printing multiple lines.

READING IN C++

```
// 1) cin - stops at the first space
char name[40];
cin >> name;
```

```
// Input : Shafiq Ur Rehman
// name  = "Shafiq"    ← only first word!
```

```
// 2) cin.getline - full line, spaces ok
char name[100];
cin.getline(name, 100);
```

```
// Input : Shafiq Ur Rehman
// name  = "Shafiq Ur Rehman"
```

01

`cin >> name`

Quick and easy — but it **stops at the first space**. Use it only for single-word input.

02

`cin.getline(name, n)`

The recommended way for full sentences. Reads until Enter, respects spaces, and caps at `n - 1` chars so it can't overflow your buffer.

!

Rule of thumb

Reaching for spaces in input? → `cin.getline`. Otherwise `cin >>` is fine.

BEGINNER · CHAR COMPARISON

COUNT THE VOWELS

Given a string, count how many vowels (a, e, i, o, u) it contains. Case-insensitive — treat 'A' and 'a' the same.

Hint · loop through, check each character against the vowel set. Don't forget uppercase.

01

Input

"Dev Weekends"

02

Expected output

4 (e, e, e, e)

03

Bonus

Also print which vowel was the most frequent.

BEGINNER · TWO POINTERS

IS IT A PALINDROME?

Check whether a string reads the same forwards and backwards. Print **Yes** or **No**.

Hint · use two indices. **i** from the start, **j** from the end.
If any pair mismatches, it's not a palindrome.

01

Input A

"madam" → **Yes**

02

Input B

"hello" → **No**

03

Bonus

Ignore case & spaces — **"Race car"** should still say **Yes**.

OPEN FLOOR

YOUR TURN

| ASK ANYTHING

QUESTIONS?

You can ask your question in the comments, and I will reply with something like this.

devweekends.com

DEV
{ WEEKENDS }

13 / 13